

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ
ФАХОВИЙ КОЛЕДЖ ІНФОРМАЦІЙНИХ СИСТЕМ І ТЕХНОЛОГІЙ
КИЇВСЬКОГО НАЦІОНАЛЬНОГО ЕКОНОМІЧНОГО УНІВЕРСИТЕТУ
імені ВАДИМА ГЕТЬМАНА

ЗВІТ

із самостійної роботи № 2

з дисципліни Конструювання програмного забезпечення

Аудит якості та рефакторинг успадкованого коду

Варіант № 2

Проект LibraryDesk

Модуль LoanManager Issue

Виконав студент групи 472

Гордєєв Дмитро Леонідович

Перевірів _____

Київ 2026

Зміст

Вихідні дані та межі роботи.....	2
1 Інвентаризація та вимірювання.....	4
2 Реєстр дефектів проектування коду.....	7
3 Страхувальна сітка характеристизаційних тестів.....	12
4 План рефакторингу.....	13
5 Виконання рефакторингів.....	14
6 Повторне вимірювання та докази поведінки.....	17
7 Реєстр технічного боргу.....	18
Висновки.....	20
Перелік використаних джерел.....	20

Вихідні дані та межі роботи

Робота виконує повний цикл аудиту якості: фіксує початковий стан, вимірює код, будує страхувальну сітку, планує та виконує малі рефакторинги, повторно вимірює результат і оцінює залишковий борг. Головний висновок: максимальна цикломатична складність знижена з 26 до 10, найдовший метод скорочено з 84 до 15 логічних рядків, а три копії формули пені замінено одним калькулятором без зміни сценарію.

Параметр	Значення
Варіант	2 LibraryDesk
Модуль-донор	LoanManager.Issue
Розрахунковий вузол	строк видачі й пеня за прострочення
Джерело	Б адаптований зріз власного проєкту
Обсяг baseline	551 фізичний LOC, 292 логічний LOC
Репозиторій	https://github.com/kuum-oss/LibraryDesk.Legacy
Теги	baseline, safety-net, after-refactoring

Важливе обмеження: методичні вказівки вимагають код іншої дисципліни або пакет викладача. Через відсутність пакета використано адаптований зріз власного LibraryDesk. Це джерело потрібно письмово погодити з викладачем до подання роботи. У звіті воно не подається як код, отриманий від викладача.

1 Інвентаризація та вимірювання

Коміт 9e9e752 з повідомленням baseline вихідний код до аудиту позначено тегом baseline. На ньому рішення збирається з нульовою кількістю помилок і попереджень. До вимірювання код не поліпшувався.

1.1 Інвентарний опис

Файл	Призначення	Мет.	Фіз. LOC	Лог. LOC
LegacyModels.cs	дані читача, книги та видачі	0	45	32
LoanManager.cs	видача, строк, пеня, стани	6	283	143
LoanRepository.cs	збереження і пошук	6	95	42
LoanReport.cs	звіти й картка читача	4	86	44
Program.cs	контрольний сценарій	0	42	31
Разом	7 типів, 5 файлів	16	551	292

1.2 Методика і метрики

Скрипт measure_metrics.py однаково обробляє baseline і кінцевий стан. СС починається з одиниці; кожен if, цикл, case, catch, логічний або тернарний оператор додає одиницю. Вкладеність рахується від тіла методу як рівня 1. Fan-out дорівнює кількості інших власних типів, згаданих класом.

Метод	LOC	СС	Вкл.	Парам.	Fan-out
LoanManager.Issue	84	26	7	8	5
LoanReport.BuildOverdueReport	19	10	4	3	3
LoanManager.CanIssue	8	10	3	2	5
LoanManager.ChangeStatus	10	9	2	2	5
LoanRepository.SumOutstandingFines	12	6	4	1	2

Метод	LOC	CC	Вкл.	Парам.	Fan-out
LoanManager.PreviewFine	9	5	3	2	5
LoanReport.BuildReaderCard	8	4	2	2	3
LoanManager.Cancel	7	4	3	2	5
LoanRepository.FindByReader	5	4	3	1	2
LoanRepository.FindByStatus	5	3	2	1	2
LoanRepository.Find	4	3	2	1	2
LoanReport.BuildInventoryReport	6	2	2	1	3
LoanRepository.Clear	4	2	2	0	2
LoanManager.DumpLog	4	2	2	0	5
LoanReport.SaveReport	4	2	2	2	3

Цикломатична складність до рефакторингу

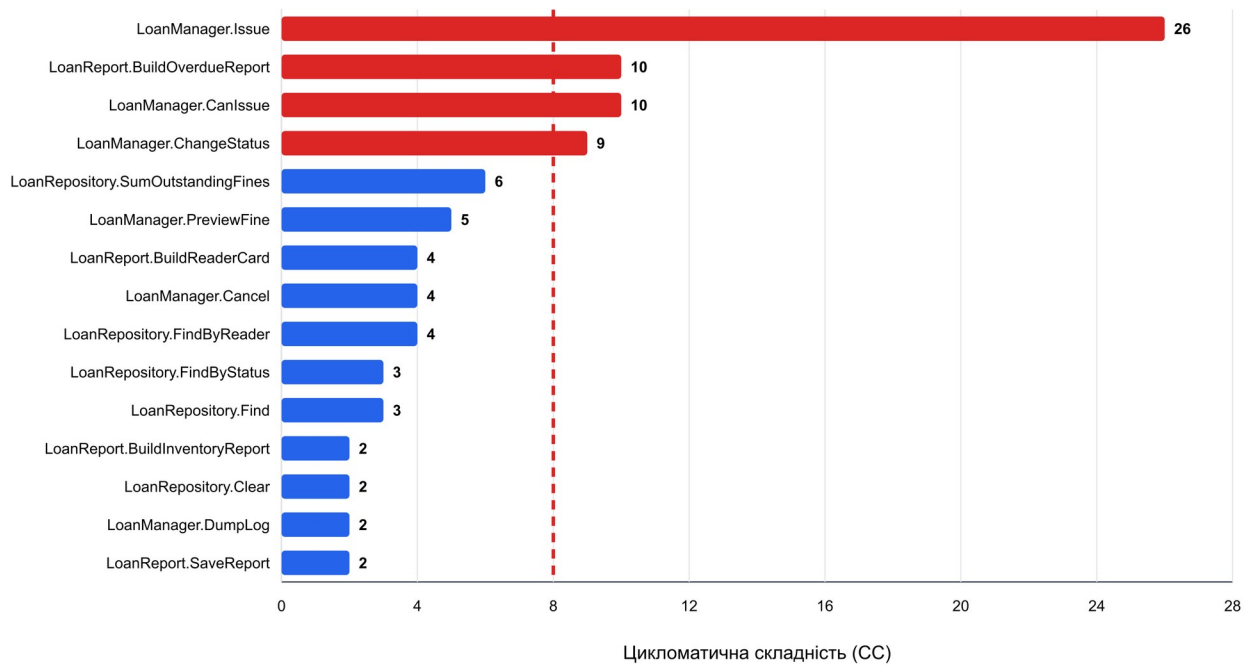


Рисунок 1 Цикломатична складність методів до рефакторингу

Порогом прийнятності обрано СС не більше 8, середню СС не більше 4, довжину методу до 40 логічних рядків і вкладеність до 3. Issue з СС 26 потребує щонайменше 26 незалежних шляхів тестування, тому його зміна без сітки є неприйнятним ризиком. Повна таблиця зберігається у docs/metrics-before.csv.

2 Реєстр дефектів проєктування коду

ID	Місце	Дефект	С	Ч	Пріор.
D-01	LoanManager.cs 14-187	Довгий метод	5	1	5
D-02	три класи	Дробовик правила пені	5	3	15
D-03	Issue 14-22	Довгий список параметрів	4	1	4
D-04	моделі і стани	Primitive obsession	4	4	16
D-05	Issue 26-79	Вкладеність 7	5	1	5
D-06	LoanManager 34-130	Магічні числа	4	4	16
D-07	_lastFine	Тимчасове поле	3	1	3
D-08	два catch	Проквітнутий виняток	5	2	10
D-09	Issue 184-185	Мертвий код	2	1	2
D-10	BuildReaderCard	Заздрість до даних	3	1	3
D-11	LegacyModels	Неінкапсульовані поля	4	4	16
D-12	Issue 20-21	Flag arguments	3	2	6

Серйозність оцінюється від 1 до 5. Частота дорівнює кількості місць, але обмежується чотирма. Пріоритет є добутком цих чисел. Реєстр містить 12 різних типів, а D-02 є профільним дефектом варіанта.

D-01 Довгий метод

84 логічні рядки, СС 26, вкладеність 7. Метод поєднує валідацію, строк, пеню, звіт, збереження та сповіщення.

```
1 public LoanResult Issue(  
2     Reader? reader,  
3     List<BookCopy>? books,  
4     DateTime issuedOn,  
5     DateTime? returnedOn,  
6     string subscription,  
7     bool sendEmail,  
8     bool printReceipt,  
9     string operatorName)
```

Лістинг 1 Доказ дефекту D-01

Гіпотеза виправлення: R01 Extract Method.

D-02 Дробовик

Формула пені повторюється у LoanManager, LoanRepository і LoanReport.
Зміна ставки потребує трьох синхронних правок.

```
1 int overdueDays = (onDate.Date - loan.DueOn.Date).Days;
2 fine = overdueDays * 2m * loan.Books.Count;
3 if (loan.Reader != null && loan.Reader.Category == "child")
4 {
5     fine = fine * 0.5m;
6 }
7 if (fine > 500m)
8 {
9     fine = 500m;
10 }
```

Лістинг 2 Доказ дефекту D-02

Гіпотеза виправлення: R09 Extract Class і R10 Move Method.

D-03 Довгий список параметрів

Вісім позиційних параметрів ускладнюють читання виклику; особливо легко переплутати дві дати й два прапорці.

```
1 Reader? reader,
2 List<BookCopy>? books,
3 DateTime issuedOn,
4 DateTime? returnedOn,
5 string subscription,
6 bool sendEmail,
7 bool printReceipt,
8 string operatorName)
```

Лістинг 3 Доказ дефекту D-03

Гіпотеза виправлення: R11 Introduce Parameter Object.

D-04 Одержимість елементарними типами

Категорія читача, група книги, абонемент і стан подані рядками; будь-яка друкарська помилка компілюється.

```

1 public string Category = "regular";
2 public string Group = "regular";
3 public string Status = "new";
4 if (subscription == "teacher")
5 {
6     days = 30;
7 }
8 loan.Status = "active";

```

Лістинг 4 Доказ дефекту D-04

Гіпотеза виправлення: R12 Replace Primitive with enum.

D-05 Надмірна вкладеність

Основний сценарій зміщений праворуч сімома рівнями, а причини відмови розпорошені по дереву умов.

```

1 if (reader != null)
2 {
3     if (!reader.IsBlocked)
4     {
5         if (books != null)
6         {
7             if (books.Count > 0)
8             {
9                 if (reader.ActiveLoans < 5)

```

Лістинг 5 Доказ дефекту D-05

Гіпотеза виправлення: R06 Guard Clauses.

D-06 Магічні числа

Ліміти, строки та ставка пені не мають предметних назв і повторюються.

```

1 if (reader.ActiveLoans < 5)
2 int days = 14;
3 days = 30;
4 days = 7;
5 days = 3;
6 fine = overdueDays * 2m * books.Count;
7 if (fine > 500m)

```

Лістинг 6 Доказ дефекту D-06

Гіпотеза виправлення: R04 Replace Magic Number with Constant.

D-07 Тимчасове поле

_lastFine заповнюється лише одним сценарієм і створює прихований стан між викликами.

```

1 private decimal _lastFine;
2 // ...
3 _lastFine = 0m;
4 if (returnedOn != null)
5 {
6     _lastFine = overdueDays * 2m * books.Count;
7     loan.Fine = _lastFine;
8 }

```

Лістинг 7 Доказ дефекту D-07

Гіпотеза виправлення: R15 Replace Temp with Query.

D-08 Проковтнутий виняток

Issue повертає успіх навіть після помилки запису. Це справжній дефект поведінки, тому його не виправляють у refactor-коміті.

```

1 try
2 {
3     _repository.Save(loan);
4 }
5 catch
6 {
7     // помилка збереження прихована
8 }

```

Лістинг 8 Доказ дефекту D-08

Гіпотеза виправлення: Окремий fix після завершення аудиту.

D-09 Мертвий код

Закоментоване обмеження не виконується і не перевіряється, але створює сумнів щодо чинного правила.

```

1 result.Success = true;
2 result.Loan = loan;
3 result.Receipt = receipt;
4
5 // Старий варіант обмежував видачу двома книгами.
6 // if (books.Count > 2) return new LoanResult();
7 return result;

```

Лістинг 9 Доказ дефекту D-09

Гіпотеза виправлення: R17 Remove Dead Code.

D-10 Заздрість до чужих даних

BuildReaderCard читає майже весь Reader і не використовує стан LoanReport.

```
1 string text = reader.Name.Trim().ToUpperInvariant();
2 text += " [" + reader.Category + "]";
3 text += " active=" + reader.ActiveLoans;
4 text += " fine=" + reader.UnpaidFine.ToString("0.00");
5 for (int i = 0; i < loans.Count; i++)
6 {
7     if (loans[i].Reader?.Id == reader.Id)
```

Лістинг 10 Доказ дефекту D-10

Гіпотеза виправлення: Move Method після поточного циклу.

D-11 Неінкапсульовані поля

Публічні поля й List дозволяють обійти будь-який інваріант стану та складу видачі.

```
1 public class Loan
2 {
3     public int Id;
4     public Reader? Reader;
5     public List<BookCopy> Books = new();
6     public DateTime DueOn;
7     public string Status = "new";
8     public decimal Fine;
9 }
```

Лістинг 11 Доказ дефекту D-11

Гіпотеза виправлення: R13 Encapsulate Field and Collection.

D-12 Булеві прапорці

Два bool утворюють чотири режими, а виклик true false не пояснює намір.

```
1 bool sendEmail,
2 bool printReceipt,
3 // ...
4 if (sendEmail)
5 {
6     Send(reader);
7 }
8 if (printReceipt)
9 {
10    Console.Write(receipt);
11 }
```

Лістинг 12 Доказ дефекту D-12

Гіпотеза виправлення: R19 Remove Flag Argument.

3 Страхувальна сітка характеристизаційних тестів

Сітка створена до першого рефакторингу й зафіксована тегом safety-net. Вона перевіряє наявну поведінку, а не бажаний дизайн: 8 сценаріїв строку й пені, 3 помилки та 2 переходи стану. Усі тести мають схему Метод Умова Результат і структуру arrange act assert.

```
1 [Fact]
2 public void Issue_FiveOverdueDaysForTwoBooks_FineIs20()
3 {
4     LoanManager sut = CreateManager();
5     LoanResult actual = sut.Issue(
6         Request(Reader(), Books(2), new DateTime(2026, 10, 1),
7             new DateTime(2026, 10, 20), SubscriptionType.Student));
8     Assert.Equal(20m, actual.Loan!.Fine);
9     Assert.Equal(LoanStatus.Overdue, actual.Loan.Status);
10 }
```

Лістинг 13 Характеризаційний тест пені

```
1 [Fact]
2 public void Issue_EmptyBookList_ReturnsEmptyError()
3 {
4     LoanManager sut = CreateManager();
5     LoanResult actual = sut.Issue(
6         Request(Reader(), [], new DateTime(2026, 10, 1), null,
7             SubscriptionType.Student));
8     Assert.False(actual.Success);
9     Assert.Equal("ERR: empty", actual.Error);
10 }
```

Лістинг 14 Характеризаційний тест помилки

```
1 [Fact]
2 public void ChangeStatus_ReturnedToActive_LeavesStateUnchanged()
3 {
4     LoanManager sut = CreateManager();
5     Loan loan = new() { Status = LoanStatus.Returned };
6     bool actual = sut.ChangeStatus(loan, LoanStatus.Active);
7     Assert.False(actual);
8     Assert.Equal(LoanStatus.Returned, loan.Status);
9 }
```

Лістинг 15 Характеризаційний тест забороненого переходу

3.1 Шов і мутаційна перевірка

Коміт f9b7560 додає необов'язковий `LoanRepository` у конструктор `LoanManager` і необов'язковий шлях у репозиторій. Значення за замовчуванням

залишають стару поведінку, а тест спрямовує файл до унікального тимчасового шляху.

Під час ручної мутації ставка 2m тимчасово замінена на 3m. Два тести стали червоними: очікуване 20 перетворилося на 30, а дитяча пеня 2 на 3. Після повернення 2m усі 13 тестів знову зелені. Це доводить, що сітка ловить зміну головного правила.

```
$ dotnet test LibraryDesk.Legacy.sln --collect:"XPlat Code Coverage"
LibraryDesk.Legacy -> bin/Debug/net8.0/LibraryDesk.Legacy.dll
LibraryDesk.Legacy.Tests -> bin/Debug/net8.0/LibraryDesk.Legacy.Tests.dll
Test run for LibraryDesk.Legacy.Tests.dll (.NETCoreApp,Version=v8.0)

Passed! Failed: 0, Passed: 13, Skipped: 0, Total: 13
Duration: 66 ms
LoanManager.cs line coverage: 74.41%
```

Рисунок 2 Результат запуску характеристичних тестів

Рядкове покриття LoanManager після завершення становить 74,41 %, гілкове 53,75 %. Ціль 60 % перевиконано.

4 План рефакторингу

Індекс обчислено як вигода поділена на ризик. За однакового індексу коротший крок стоїть вище.

№	ID	Рефакторинг	В	Р	хв	Індекс
1	D-06	R04 константи	5	1	15	5,00
2	D-05	R06 guard clauses	5	1	20	5,00
3	D-07	R15 запит	4	1	15	4,00
4	D-01	R03 змінна	3	1	10	3,00
5	D-02	R09 FineCalculator	5	2	35	2,50
6	D-01	R01 методи	5	2	40	2,50

№	ID	Рефакторинг	В	Р	хв	Індекс
7	D-09	R17 мертвий код	2	1	5	2,00
8	D-03	R11 IssueRequest	4	2	35	2,00
9	D-11	R13 інкапсуляція	4	2	45	2,00
10	D-02	R10 Move Method	5	3	50	1,67
11	D-04	R12 enum	5	3	50	1,67
12	D-12	R19 remove flags	3	3	40	1,00

4.1 Правила безпеки

1. Один тип рефакторингу дорівнює одному коміту; зміну поведінки не змішувати.
2. Після кожного кроку запускати build і всі 13 тестів.
3. Крок довший за 20 хвилин ділити на менші частини.
4. Якщо тести червоні понад 10 хвилин, відновити файли поточного кроку.
5. Очікувані значення не змінювати; при зміні сигнатури правити лише виклик.
6. Перед комітом перевіряти diff, після коміту записувати короткий хеш.

5 Виконання рефакторингів

№	Код	Зміна	Коміт	Passed
1	R04	константи	06e0df5	13
2	R06	guard clauses	8d7d33d	13
3	R15	запит пені	abc0150	13
4	R03	умова в змінній	bb1d1e0	13
5	R09	FineCalculator	6d44db9	13
6	R01	етапи Issue	e97f88a	13
7	R17	мертвий код	c894224	13
8	R11	IssueRequest	fbd9a8b	13
9	R13	інкапсуляція	531d7e4	13
10	R10	перенесення пені	1b97e4d	13

№	Код	Зміна	Коміт	Passed
11	R12	enum	9ac5808	13
12	R19	без прапорців	186691b	13
13	R06	охоронна гілка звіту	4e924a6	13

Історія містить 13 refactor-комітів і 12 різних типів. R06 повторено у другому незалежному місці. Обов'язкові R09 і R10 виконані у комітах 6d44db9 та 1b97e4d. Тег after-refactoring стоїть на останньому рефакторингу, а документація додана пізніше.

5.1 Приклади було і стало

```

1 if (reader != null)
2 {
3     if (!reader.IsBlocked)
4     {
5         if (books != null)
6         {
7             if (books.Count > 0)
8             {
9                 // основний сценарій

```

Лістинг 16 Було вкладене дерево валідації

```

1 if (reader == null) return "ERR: reader";
2 if (reader.IsBlocked) return "ERR: blocked";
3 if (books == null) return "ERR: null-books";
4 if (books.Count == 0) return "ERR: empty";
5 if (reader.ActiveLoans >= MaximumActiveLoans
6     && reader.Category != ReaderCategory.Staff)
7     return "ERR: limit";

```

Лістинг 17 Стало охоронні умови

```

1 fine = overdueDays * 2m * loan.Books.Count;
2 if (loan.Reader?.Category == "child")
3 {
4     fine *= 0.5m;
5 }
6 if (fine > 500m)
7 {
8     fine = 500m;
9 }

```

Лістинг 18 Було одна з трьох формул пені

```

1 public decimal Calculate(Loan loan, DateTime onDate)
2 {
3     if (onDate.Date <= loan.DueOn.Date) return 0m;
4     int days = (onDate.Date - loan.DueOn.Date).Days;
5     decimal fine = days * FinePerBookPerDay * loan.Books.Count;
6     if (loan.Reader?.Category == ReaderCategory.Child)
7         fine *= ChildFineRate;
8     return Math.Min(fine, MaximumFine);
9 }

```

Лістинг 19 Стало єдиний FineCalculator

```

1 public LoanResult Issue(
2     Reader? reader, List<BookCopy>? books,
3     DateTime issuedOn, DateTime? returnedOn,
4     string subscription, bool sendEmail,
5     bool printReceipt, string operatorName)

```

Лістинг 20 Було вісім параметрів

```

1 public sealed record IssueRequest(
2     Reader? Reader,
3     List<BookCopy>? Books,
4     DateTime IssuedOn,
5     DateTime? ReturnedOn,
6     SubscriptionType Subscription,
7     string OperatorName);

```

Лістинг 21 Стало об'єкт параметрів

```

1 LoanResult result = manager.Issue(
2     request,
3     sendEmail: true,
4     printReceipt: false);

```

Лістинг 22 Було прапорці

```

1 LoanResult result = manager.IssueAndNotify(request);
2
3 public LoanResult IssueAndNotify(IssueRequest request)
4 {
5     LoanResult result = Issue(request);
6     if (result.Success) SendEmail(request.Reader!);
7     return result;
8 }

```

Лістинг 23 Стало явний сценарій

6 Повторне вимірювання та докази поведінки

Метрика	До	Після	Зміна	%
Логічний LOC	292	326	+34	+11,64 %
Методів	16	28	+12	+75,00 %
Максимальна СС	26	10	-16	-61,54 %
Середня СС	5,81	3,04	-2,77	-47,68 %
Найдовший метод	84	15	-69	-82,14 %
Макс. вкладеність	7	3	-4	-57,14 %
Клони 6+ рядків	1	0	-1	-100 %
Макс. fan-out	5	11	+6	+120 %
Тестів	0	13	+13	—
Покриття LoanManager	0 %	74,41 %	+74,41 п.п.	—

Максимальна СС лишилася 10 у CanIssue, але знизилася на 61,54 %, що перевищує допустиму альтернативу 40 %. LOC зріс на 11,64 % через IssueRequest, FineCalculator і enum. Fan-out зріс до 11, бо нові типізовані залежності рахуються окремо; це зафіксовано як TD-03.

Цикломатична складність до і після рефакторингу

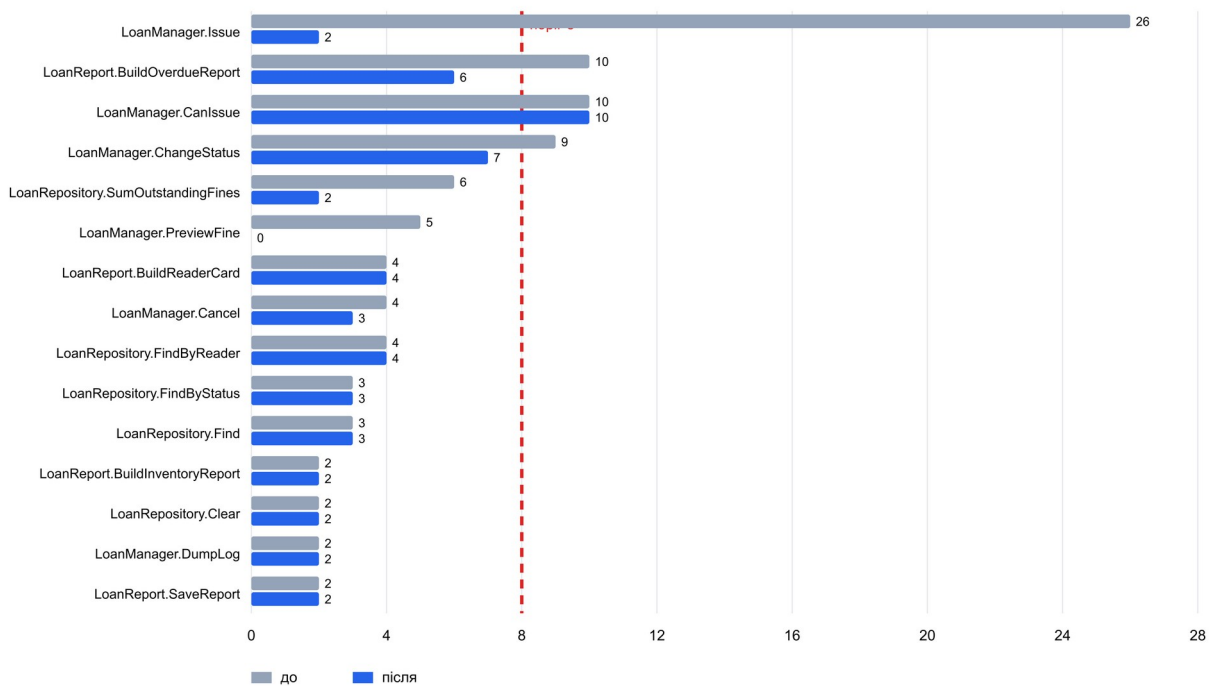


Рисунок 3 Цикломатична складність до і після

6.1 Докази незмінності

1. Тести після safety-net змінювалися лише разом із сигнатурами R11, R12 і R19. Числові очікування не змінювалися.
2. На baseline з тестами safety-net Passed 13, Failed 0; на after-refactoring Passed 13, Failed 0.
3. out-before.txt і out-after.txt ідентичні; diff повертає код 0 без рядків відмінності.

```

1 $ git log --oneline safety-net..after-refactoring -- Legacy.Tests
2 186691b refactor(LoanManager): R19 вилучено прапорці
3 9ac5808 refactor(models): R12 введено переліки
4 fbd9a8b refactor(LoanManager): R11 введено IssueRequest
5
6 $ git diff --no-index docs/out-before.txt docs/out-after.txt
7 # виводу немає, код завершення 0

```

Лістинг 24 Докази історії тестів і однакового сценарію

7 Реєстр технічного боргу

Тіло оцінено PERT за формулою O плюс $4M$ плюс P , поділене на 6.

ID	Опис	О/М/Р	Тіло	год/міс	міс
TD-01	Проквітнуті винятки	1/2/4	2,17	1,00	2,17
TD-02	Немає тестів сховища і звіту	2/4/7	4,17	1,50	2,78
TD-03	Fan-out LoanManager 11	2/3/6	3,33	1,00	3,33
TD-04	CanIssue CC 10	1,5/2,5/4	2,58	0,75	3,44
TD-05	Пряма залежність від File	2/4/8	4,33	1,25	3,46
TD-06	Feature envу картки	1/1,5/3	1,67	0,50	3,34

Сумарне тіло боргу 18,25 год. Обслуговування 6,00 год на місяць.
Загальна окупність $18,25 \div 6,00 = 3,04$ місяця.

7.1 План погашення

№	ID	Дія	Коли	Критерій
1	TD-01	не ковати IOException	до ЛР-7	окремий зелений тест
2	TD-02	тести сховища і звітів	ЛР-7	покриття $\geq 70\%$
3	TD-03	розділити координату	до ЛР-8	$\text{fan-out} \leq 7$
4	TD-06	ReaderCardPresenter	тиждень 10	немає feature envу
5	TD-04	єдине правило CanIssue	тиждень 10	$\text{CC} \leq 5$
6	TD-05	адаптер I/O	до ЛР-8	атомарний запис

TD-01 і TD-02 мають окупність до трьох місяців, тому стоять першими. TD-01 не виправлено під час рефакторингу навмисно: передавання винятку змінить зафіксовану поведінку й потребує окремого фік-коміту. Усі шість пунктів позначені TODO безпосередньо в коді.

Висновки

1. Початковий модуль містив 16 методів, а LoanManager.Issue мав 84 логічні рядки, СС 26 і вкладеність 7.
2. Сітка з 13 тестів створена до змін, забезпечила понад 60 % покриття й виявила ручну мутацію ставки двома падіннями.
3. Профільний дробовик усунено обов'язковими Extract Class і Move Method: три формули замінено FineCalculator.Calculate.
4. Найбільший ефект на витрачений час дали guard clauses, які знизили ризик наступного Extract Method.
5. Виконано 12 різних типів рефакторингу окремими комітами; на кожному кроці проходили 13 тестів.
6. Максимальна СС знизилася на 61,54 %, найдовший метод на 82,14 %, вкладеність до 3, а клони до нуля.
7. Залишковий борг становить 18,25 год і коштує 6 год на місяць; перші дії стосуються винятків і прогалин покриття.

Перелік використаних джерел

1. Фаулер М. Рефакторинг Поліпшення наявного коду. Друге видання.
2. Фезерс М. Ефективна робота з успадкованим кодом.
3. Microsoft Learn. Документація C# і .NET 8.
4. xUnit.net. Документація фреймворку модульного тестування.
5. Coverlet. Документація зі збирання покриття .NET.
6. Методичні вказівки до самостійної роботи № 2 Аудит якості та рефакторинг успадкованого коду.